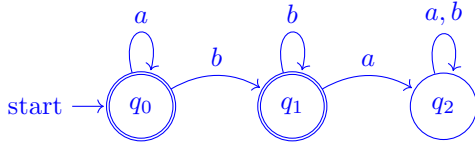


Moed A - Solution

Question 1

a. Consider the CFG G with the rules $S \rightarrow aSb|aS|Sb|\epsilon$. Is $L(G) \in REG$? If so, draw a minimal DFA for $L(G)$ (no need to prove correctness or minimality). Otherwise, prove that $L(G) \notin REG$.

$L(G)$ is regular and the following is the minimal DFA for $L(G)$:



b. For a language L over Σ and words $x, y \in \Sigma^*$, we say that $z \in \Sigma^*$ is a *separating suffix* between x and y if it is **not** that case that $x \cdot z \in L \iff y \cdot z \in L$. Let $L_{x,y}$ denote the language of all separating suffixes between x and y . Claim: If $L \in REG$, then for all $x, y \in \Sigma^*$, we have that $L_{x,y} \in REG$.

The claim is correct. Proof: assume that $L \in REG$ and let $\mathcal{A} = \langle Q, \Sigma, q_0, \delta, F \rangle$ be a DFA for L . Let x and y be words in Σ^* . Denote $q_x = \delta^*(q_0, x)$ and $q_y = \delta^*(q_0, y)$. For every word $z \in \Sigma^*$, it holds that $\delta^*(q_0, xz) = \delta^*(q_x, z)$ and thus we have that $xz \in L$ iff $\delta^*(q_x, z) \in F$. Similarly, $yz \in L$ iff $\delta^*(q_y, z) \in F$. Therefore, to prove that $L_{x,y}$ is regular, we need to show the existence of a DFA $\mathcal{B}$ that accepts exactly all words z with:

$$\delta^*(q_x, z) \in F \wedge \delta^*(q_y, z) \notin F$$

Or

$$\delta^*(q_x, z) \notin F \wedge \delta^*(q_y, z) \in F$$

To do this, we need to simultaneously run the input word, z , from q_x and q_y and accept iff exactly one of the runs is accepting. Hence, the following product automaton, of $\mathcal{A}$ with itself, recognizes $L_{x,y}$:

$$\mathcal{B} = \langle Q \times Q, \Sigma, \langle q_x, q_y \rangle, \delta_B, ((Q \setminus F) \times F) \cup (F \times (Q \setminus F)) \rangle$$

Indeed, it holds that

$$z \in L(\mathcal{B}) \iff \delta_B^*(\langle q_x, q_y \rangle, z) \in ((Q \setminus F) \times F) \cup (F \times (Q \setminus F))$$

$$\iff (\delta^*(q_x, z), \delta^*(q_y, z)) \in ((Q \setminus F) \times F) \cup (F \times (Q \setminus F))$$

$$\iff \text{The run of } q_x \text{ on } z \text{ is accepting OR the run of } q_y \text{ on } z \text{ is accepting. But not both}$$

Note: we've used the fact that $\delta_B^*(\langle q_x, q_y \rangle, z) = (\delta^*(q_x, z), \delta^*(q_y, z))$ for every $z \in \Sigma^*$. We've seen this simple fact in class by induction on $|z|$.

c. Recall that for a word $w = w_1, w_2, \dots, w_k$, with $w_i \in \Sigma$, we let $rev(w) = w_k, \dots, w_2, w_1$. Let $L = \{xy : x = y \text{ or } x = rev(y)\}$ be a language over $\Sigma = \{0, 1\}$. Is $L \in REG$? Is $L \in CFL$.

We'll prove by the pumping lemma of CFLs that $L \notin CFL$ and thus also conclude that $L \notin REG$: assume by the way of contradiction that $L \in CFL$ and let $p > 0$ be the pumping constant. Consider the word $w = 0^p 1^p 0^p 1^p$. Clearly $w \in L$ and $|w| > p$, so according to the pumping lemma, there exist $u, v, x, y, z \in \Sigma^*$ such that $w = uvxyz$, $|vxy| \leq p$, $|vy| > 0$ and for all $i \geq 0$ it holds that $uv^i xy^i z \in L$. We split into cases:

- Suppose that vxy is in the first half of w : the second half of $uv^2 xy^2 z$ starts with 1. In addition, $uv^2 xy^2 z$ starts with 0 and ends with 1. Hence $uv^2 xy^2 z$ is neither of the form ww nor of the form ww^{rev} .

- Suppose that vxy is in the second half of w : the first half of uv^2xy^2z ends with 0. In addition, uv^2xy^2z starts with 0 and ends with 1. Hence uv^2xy^2z is neither of the form ww nor of the form ww^{rev} .
- Suppose that vxy touches both the first and second half of w : uv^0xy^0z is of the form $0^p1^m0^n1^p$, where at least one of m or n does not equal p . Hence, uv^0xy^0z is not of the form ww . In addition, uv^0xy^0z starts with 0 and ends with 1 and thus it is also not of the form ww^{rev} .

Question 2

a. Claim: If $L \in RE$ and $L \leq_m \bar{L}$, then $L \in R$. (recall that $\bar{L} = \Sigma^* \setminus L$).

The claim is correct. Proof: we are given that $L \in RE$ which is equivalent to $\bar{L} \in coRE$. Now by applying the reduction theorem on $L \leq_m \bar{L}$, we get that $L \in coRE$. All in all, we have that $L \in RE \cap coRE$ and since $R = RE \cap coRE$, we are done.

b + c. For a finite alphabet Σ , let

$$L_2 = \{\langle M_1, M_2 \rangle : M_1 \text{ and } M_2 \text{ are Turing Machines and } |L(M_1)| \leq |L(M_2)|\}$$

Note that $\aleph_0 \leq \aleph_0$.

b. Claim: $L_2 \in RE$.

The claim is not correct. Proof: we show that $\overline{A_{TM}} \leq_m L_2$. Note that this is sufficient. Indeed, since $\overline{A_{TM}} \notin RE$ then, by the reduction theorem, $\overline{A_{TM}} \leq_m L_2$ implies that $L_2 \notin RE$.

Construction: Given input $\langle M, w \rangle$, the reduction outputs $\langle M_1, M_2 \rangle$ where M_2 is a fixed machine that recognizes $\emptyset$ and M_1 works as follows. Given input x , M_1 simulates the run of M on w and answers the same. That is, if M accepts M_1 accepts, if M rejects M_1 rejects and clearly, if M does not halt then so is M_1 .

Correctness: If $\langle M, w \rangle \in \overline{A_{TM}}$, that is, if M does not accept w . We get for every input x that M_1 also either does not halt on or rejects x . Hence $L(M_1) = \emptyset = L(M_2)$, implying that $|L(M_1)| \leq |L(M_2)|$ and thus $\langle M_1, M_2 \rangle \in L_2$. Conversely, assume that $\langle M, w \rangle \notin \overline{A_{TM}}$, that is, M accepts w . Hence M_1 accepts every input word x , implying that $|L(M_1)| = \infty > 0 = |L(M_2)|$ and thus, $\langle M_1, M_2 \rangle \notin L_2$.

Computability: Given $\langle M, w \rangle$, it is easy to construct an encoding of a TM M_1 that, for every input x , simulates the run of M on w . Indeed, simulating M on w can be done by a universal machine. In addition, concatenating to the encoding of M_1 , an encoding of a constant TM that recognizes $\emptyset$ is also computable.

c. Claim: $L_2 \in coRE$.

The claim is not correct. Proof: we show that $A_{TM} \leq_m L_2$. Note that this is sufficient. Indeed, since $A_{TM} \notin coRE$ then, by the reduction theorem, $A_{TM} \leq_m L_2$ implies that $L_2 \notin coRE$.

Construction: Given input $\langle M, w \rangle$, the reduction outputs $\langle M_1, M_2 \rangle$ where M_1 is a fixed machine that recognizes Σ^* and M_2 works as follows. Given input x , M_2 simulates the run of M on w and answers the same. That is, if M accepts M_2 accepts, if M rejects M_2 rejects and clearly, if M does not halt then so is M_2 .

Correctness: If $\langle M, w \rangle \in A_{TM}$, that is, if M accepts w . We get that M_2 accepts every input word x . Hence $L(M_2) = \Sigma^* = L(M_1)$, implying that $|L(M_1)| \leq |L(M_2)|$ and thus $\langle M_1, M_2 \rangle \in L_2$. Conversely, assume that $\langle M, w \rangle \notin A_{TM}$, that is, M does not accept w . In this case, for every input x , M_2 also does not accept. Hence $L(M_2) = \emptyset$, implying that $|L(M_1)| = \infty > 0 = |L(M_2)|$ and Thus $\langle M_1, M_2 \rangle \notin L_2$.

Computability: Given $\langle M, w \rangle$, it is easy to construct an encoding of a TM M_2 that, for every input x , simulates the run of M on w . Indeed, simulating M on w can be done by a universal machine. In addition, constructing an encoding of a constant TM that recognizes Σ^* is also computable.

Note: it may be tempting to try and do the following reduction from $ALL_{TM} = \{\langle M \rangle : L(M) = \Sigma^*\}$ to L_2 : for input $\langle M \rangle$ the reduction outputs $\langle M, M' \rangle$ where M' is some fixed machine with $L(M') = \Sigma^*$. This reduction does not work because if $L(M) \neq \Sigma^*$ then this does not necessarily mean that $L(M)$ is finite.

Question 3

a. Recall that $Clique = \{\langle G, k \rangle : \text{the graph } G \text{ has a set of vertices of size } k \text{ such that all pairs of vertices in the set have an edge between them}\}$.

Claim: if $Clique \in coNP$, then $NP = coNP$.

The claim is correct. Proof: let A and B be languages such that $A \leq_p B$. We have seen in Ex10 that if A is NPC and $B \in coNP$ then $NP = coNP$. In this question we are given $Clique$ which we know from class to be an NPC problem. We are also given that $Clique \in coNP$. Hence, if we use the fact that $Clique \leq_p Clique$ and¹ use the claim mentioned above for $A = B = Clique$, we get that $NP = coNP$. In simpler words, the question was almost taken from Ex10.

b. In the language $ApSum$, the input includes a target sum S and a set of numbers. All numbers are natural and are given in binary. The input is in the language if there is a subset of the numbers whose sum is at distance at most 10 from S . Formally, $ApSum = \{(S, \{a_1, \dots, a_t\}) : \exists U \subseteq \{1, \dots, t\} \text{ such that } S - 10 \leq \sum_{i \in U} a_i \leq S + 10\}$. Prove membership of $ApSum$ in P, NPC, or co-NPC.

$ApSum$ is NPC:

1. $ApSum \in NP$: given input $\langle S, \{a_1, \dots, a_m\} \rangle$. The witness that $x = \langle S, \{a_1, \dots, a_m\} \rangle \in ApSum$ is a set of indices $U \subseteq \{1, \dots, m\}$ with $S - 10 \leq \sum_{i \in U} a_i \leq S + 10$. The verifier gets as input $\langle x, U \rangle$ and checks whether $S - 10 \leq \sum_{i \in U} a_i \leq S + 10$ or not, as follows. First, it checks whether U is a subset of $\{1, \dots, m\}$. Then it sums the elements in $\{x_i : i \in U\}$ and subtracts S from that sum. All in all, the verifier has computed $r = \sum_{i \in U} a_i - S$. Finally, the verifier checks whether $-10 \leq r$ and $r \leq 10$. Clearly, all this can be done in time polynomial in $|x|$.
2. $ApSum$ is NP-Hard: we show that² $SubsetSum \leq_p ApSum$.

Construction: Given a set of natural numbers $R = \{x_1, \dots, x_m\}$ and a natural target t as input for $SubsetSum$, the reduction computes $11x_1, 11x_2, \dots, 11x_m$ and $11t$ and outputs the set $F = \{11x_1, \dots, 11x_m\}$ and the target sum $S = 11t$.

Correctness: If $\langle R = \{x_1, \dots, x_m\}, t \rangle \in SubsetSum$, then there exists a subset $M \subseteq \{1, \dots, m\}$ with $\sum_{i \in M} x_i = t$, which implies that $\sum_{i \in M} 11x_i = 11t = S$. That is, there exists a subset of $U \subseteq \{1, \dots, m\}$ with $\sum_{i \in U} 11x_i = S$ and thus $\langle F, S \rangle \in ApSum$. Conversely, if $\langle R = \{x_1, \dots, x_m\}, t \rangle \notin SubsetSum$, then for every subset $M \subseteq \{1, \dots, m\}$ it holds that $\sum_{i \in M} x_i \neq t$. Therefore³, $|(\sum_{i \in M} x_i) - t| \geq 1$ which implies that $|(\sum_{i \in M} 11x_i) - 11t| \geq 11$. That is, for every subset $U \subseteq \{1, \dots, m\}$ we have that $|(\sum_{i \in U} 11x_i) - S| \geq 11$ and thus $\langle F, S \rangle \notin ApSum$.

Runtime: Let i denote the number of letters used to represent the largest number in the input. Clearly $i \leq |\langle R, t \rangle|$. The runtime of the reduction is bounded by the time for multiplying each of the input numbers by 11. Since 11 is a constant, this is polynomial in $(m + 1) \cdot i$. and since $m \leq |\langle R, t \rangle|$, we get that the total runtime is polynomial in $|\langle R, t \rangle|$.

We conclude from 1. and 2. that $ApSum$ is NPC.

c. Claim: if $L_1 \in R, L_2 \in P$ and L_1, L_2 are both not empty and not Σ^* , then there is a **polynomial** reduction $L_1 \leq_p L_2$.

The claim is not correct. Proof: assume by contradiction that the claim is correct. By the reduction theorem we get that $R \subseteq P$ and thus $R = P$, a contradiction to the fact that⁴ $P \neq EXP \subseteq R$.

¹Or we can use instead the fact that $SAT \leq_p CLIQUE$

²You've seen in class that the natural numbers (binary encoded) version of $SubsetSum$ is NPC

³This follows from the fact that we're dealing with natural numbers

⁴You've seen this as a conclusion of the time hierarchy theorem